

Project Improvements Using a UI-Oriented Approach to Acceptance Criteria and BDD

By Tim Fox

Project improvements using a UI-oriented approach to Acceptance Criteria and BDD (Behavior-Driven Development), primarily from a Test Automation perspective.

Acceptance Criteria in Requirements

- Many projects are adding acceptance criteria (AC) to each requirement.
- Primary purpose of AC is to define the exact conditions that must be met for a requirement or user story to be considered "complete" and ready for stakeholders such as Dev and Test.
- AC act as a clear, pass/fail checklist that eliminates ambiguity and ensures alignment between the business and development teams.
- Clarifies expected outcomes: Translates high-level business goals into specific, testable behaviors.
- Prevents scope creep: Sets firm boundaries on what needs to be built now versus what can wait for later iterations.
- Guides Quality Assurance (QA): Provides a concrete foundation for developers and testers to design acceptance tests and verify functionality.
- Aligns Stakeholders: Ensures everyone has a shared understanding of what the end-user requires, reducing the need for costly rework.

Acceptance Criteria: Project Stakeholders

How AC is created, maintained and used on the project:

- Product team created draft requirements, including draft AC, in Jira tickets.
- *Small* group (consisting of one person from each of the following teams: Product, Dev, Test, Project Management) improved each requirement (aka ticket), including the AC, during formal Grooming Sessions within Sprints. We had 2 x 2-hour grooming sessions per 2-week Sprint.
- Grooming sessions were applied to the project's backlog of tickets. The sessions ensured that each groomed ticket in the backlog was sufficiently defined (including its AC) to be incorporated into a subsequent Sprint.
- If small group felt that ticket could not be completed within a 2-week Sprint, ticket was split into multiple, linked tickets.
- *Big* group (consisting of pretty much everyone) held regular grooming sessions to review each small-group groomed ticket in the backlog. Reviewing involved: suggesting improvements, voting whether the ticket was fully defined, and voting on its size (story points). If major re-work was needed to a ticket, re-work would be done by small group.
- During Sprint Planning sessions (one session at the beginning of each new Sprint), the big group choose which fully-groomed tickets from the backlog should be moved to the next Sprint for development by Dev and Test.
- During the Sprint, each ticket was worked in unison by Dev and Test, with Dev providing early, incomplete releases to Test so that both parties could work on the same ticket in parallel. Test team produced only automated tests.
- Dev and Test work was peer-reviewed, and signed off after any re-work and re-reviews.
- Signed-off, completed tickets were demoed to Product by Dev and/or Test. Jira ticket set to Complete by Product if satisfied.

Can Manual Testers Write Automated Tests?

- The project was a PoC (Proof of Concept) project.
- A major goal was to determine whether company's existing team of manual testers (who have little programming knowledge) could develop the executable automated tests from requirements' AC without programming training.
- The idea was that the manual test team would use BDD/Cucumber to develop the automated test cases from AC using just Gherkin and they would not need to write any (or very little) of their own Python code.
- SDETs support them by building the overall test framework in Python.
- SDETs also develop some initial automated test cases from requirements' AC in Gherkin, and the manual testers would subsequently use those initial automated test cases as templates.
- Another goal was to determine the ratio of SDETs to manual testers needed when this approach was used by subsequent projects.

Typical Acceptance Criteria: User Login

- **Feature: A registered user signs in to the application**

- **Scenario: Sign in with valid credentials**

Given the user is on the login page

When they enter a valid username and password and select “Login”

Then they are taken to their dashboard

- **Scenario: Sign in with invalid credentials**

Given the user is on the login page

When they enter an incorrect username or password and select “Login”

Then the error “Invalid username or password” is shown and they stay on the login page

This is the typical BDD specification for a login feature, expressing acceptance criteria's behaviour in BDD's Given-When-Then form. This AC would be included in the Jira ticket for the Login requirement/ticket/feature.

Creating an Automated Test from AC

Creating an automated test in Gherkin from a requirement's BDD-styled AC involves:

- Creating a *.feature* file for the requirement, e.g. the *login.feature* file
- For each Scenario in the AC, adding a Scenario clause (in Gherkin) to the feature file.
- For each Given/When/Then step in a Scenario, add one or more steps (in Gherkin) to the feature file.
- Each Gherkin step in the feature file maps to an actual function written in a real programming language such as Python. Those actual functions are known as the *step definition functions*.
- When the feature file is executed, the chosen BDD framework (e.g. Cucumber) executes each Scenario in turn by executing the steps in that Scenario.
- Each Gherkin step is executed by Cucumber finding and invoking the step definition function corresponding to the step.
- The Gherkin language's syntax allows parameters to be included in a step, and Cucumber passes the step's parameters (if any) to the invoked step definition function.
- At run time, the test framework finds and executes the *.feature* files.
- If a step fails, i.e. if the invoked step definition function fails, then the Scenario fails.

Problems using typical style AC

Problems encountered when AC is written in typical behaviour-oriented BDD style:

- The AC in the previous Login scenarios encouraged SDETs to develop initial tests in a manner that proliferated many individual, unique-purpose Python step definition functions, with each being individually programmed in Python. That style that could not be mimicked by the company's existing team of manual testers since such programming would be beyond their ability.
- For example, the When step is similar in both Login scenarios (both steps enter values into text fields), with the only diff being the values to be entered into the fields. But the AC as written seemed to encourage SDETs to develop 2 distinct step definition functions: one hard-coded to enter valid data and the other hard-coded to enter invalid data.
- Similarly, the Given step in both Login scenarios encouraged SDETs to develop a step and step definition function pair that was hard coded to verify that the login page was displayed. If the project had 10 pages, this style of AC seemed to encourage the SDETs to develop 10 unique step and step definition function pairs, one for each page.
- What was needed was a way to encourage SDETs to develop much more re-usable steps and step definition functions, i.e. ones once developed by the SDETs in Python could be re-used in many scenarios, including in future Gherkin scenarios developed by the manual testers. For example, a single step and step definition function pair that could handle entering text in any named text field, and another pair that could verify that any named page is currently being displayed, etc.

Problems using typical style AC continued ...

- **A 2nd problem was ticket scope creep. The Dev team would develop code for edge cases for which no AC had been specified and hence that code wasn't tested by Test and wasn't verified by Product.**
- **Hence a rule was introduced that Dev would only write code for an edge case if there was an explicit scenario for that edge case in the ticket's AC.**
- **The CI/CD pipeline would fail if code coverage was too low, thus preventing Dev introducing code that had no corresponding test.**
- **This drastically reduced both scope creep and ticket overruns during a Sprint.**

UI-Oriented AC: User Login (valid creds)

- **Feature: A registered user signs in to the application**
- **Scenario: Sign in with valid credentials**
 - Given** the “Login” page is displayed
 - When** I populate the “Login” page with the following:
 - | Username | johndoe |
 - | Password | somepassword |
 - And** I click the “Login” button
 - Then** the “Login” page is not displayed
 - And** the “Home” page is displayed
 - And** the “Home” page’s “Username” displays “johndoe”
 - And** the “AuditLog” file contains “johndoe logged in” at location “NearTopOfFile”

This is a more UI-oriented BDD specification for the AC for the valid-credentials login scenario.

UI-Oriented AC: User Login (invalid creds)

- **Feature: A registered user signs in to the application**
- **Scenario: Sign in with invalid credentials**

Given the “Login” page is displayed

When I populate the “Login” page with the following:

| Username | johndoe |

| Password | badpassword |

And I click the “Login” button

Then the “Login” page is displayed

And the “Home” page is not displayed

And the “Error” dialog is displayed

And the “Error” dialog’s “Message” displays “Invalid username or password”

And the “AuditLog” file does not contain “ logged in” at location “NearTopOfFile”

This is a more UI-oriented BDD specificjohn doeation for the AC for the invalid-credentials login scenario.

Library of highly re-usable step def functions

UI-oriented step / step definition function pairs facilitate:

- Use parameters (double-quoted strings) in steps to pass names, values, etc. to the step definition function, highly increasing step reuse across many scenarios.
- For example, the step “When I click the {page_name} page’s {button_name} button” can be used to click any button on any page.
- Similarly, the step “When I populate the {page_name} page with the following:”, followed by the data bundle (formally a Gherkin table) listing the control-name / control-value pairs, can be used to enter many different value sets into different input field combinations for many different pages.
- This style is much more like a traditional manual test with each test step detailing what action the tester is to perform (typically) through the UI, and what the application’s expected reaction should be to that action (typically) from a UI perspective.

Library of re-usable step defs continued 1...

- Not all reusable steps have to be through the application's UI.
- For example, a human tester could use any text editor to verify the contents of the AuditLog file (see last step in both scenarios). This reusable step could be used to verify the partial contents of any text file (and not just the AuditLog file).
- Reusable steps can also cover REST API testing, e.g.:

Scenario: Test by sending a REST API request and verifying the response.

When I send an API request with the following:

Method	URL
GET	some url

Then an API response is received with the following:

StatusCode	Body
200	{ expected JSON body }

, where Gherkin tables are used to provide the details of the request to be sent and the expected response.

- In API tests, the *When's* step definition function is coded to send the request and store the response, and that response is subsequently consumed and verified by the *Then's* step definition function.

Library of re-usable step defs continued 2...

- **Note that many step / step definition function pairs have an opposite corresponding pair, e.g. that a control “is displayed” vs “is not displayed”, and that a control is displaying a specified value vs is not displaying that value, etc.**
- **The aim is to gradually build up a library of highly reusable step / step definition function pairs so that scenarios can be developed 100% in Gherkin without any new real code (e.g. Python code) needing to be written.**
- **This dramatically improves test development speed all-round, be it for SDETs, manual testers writing automated tests, Dev, etc.**
- **In addition to the manual test team, I even trained Product and Scrum Master to develop tests so as to improve their AC writing from a Test perspective.**
- **We (mostly myself) gradually built up a library of over 100 highly reusable step / step def function pairs, covering UI interactions with text fields, buttons, checkboxes, radio buttons, dropdowns, grids, graphs, context menus, page navigation, etc., plus reusable steps for file existence and file contents.**

POM (Page Object Model)

A few test automation implementation details:

- **For each page/dialog/tab of the AUT (Application Under Test), the test framework defines a separate POM in its own separate Python file), i.e. one POM per significant chunk of UI.**
- **The POM specifies all constant (i.e. non-runtime) characteristics for that UI chunk, including:**
 - **The name by which the chunk is referred to in test steps, e.g. the “Login” page or the “Error” dialog.**
 - **The set of child controls in the UI chunk, i.e. all the buttons, text fields, dropdowns, etc.**
 - **For each child control:**
 - **The name by which the child control is referred to in test steps, e.g. the “Add” button, the “FirstName” text field, the “Gender” dropdown, etc.**
 - **The child control’s type, e.g. button, text field, dropdown, checkbox, grid, tree, menu, breadcrumb, etc.**
 - **The child control’s locator and location type, i.e. how to find the control at runtime, e.g. Xpath string, Xname for WPF, etc.**
 - **The child control’s format type (if applicable), i.e. whether the control’s displayed value contains format chars added automatically by the app rather than entered through the UI, e.g. phone numbers, times and dates, etc. The test framework can then automatically strip out (or add) formatting chars when comparing the control’s displayed value against its entered (and hence) expected value.**
- **When the test framework loads at the start of a test execution session, the framework registers into a map the POM details for all POMs. That map is then available at run time when test steps are executed.**

POM continued 1...

- **Most UI-oriented steps in the reusable steps library take the name of a POM as either:**
 - **An explicit step parameter, e.g. ‘Then the “Login” page is displayed’, where “Login” is the explicit name for the POM to be used, or**
 - **An implied value determined by context, e.g. ‘When I click the current page’s “Add” button’, where the test framework determines the name of the current page and uses that as the name of the POM, or**
 - **A default POM name value set explicitly by a previous step in the scenario, e.g. ‘When I click the “Login” button’ is sufficient since the populate step before it mentioned the “Login” POM (thus making it the default POM for the scenario).**

POM continued 2...

- **Similarly, most UI-oriented steps in the reusable steps library can also take the name of the application as either:**
 - **An explicit step parameter, e.g. ‘Then the “Myapp” application’s “Login” page is displayed’, where “MyApp” is the explicit name used to find the application at run time, or**
 - **A default app name value set explicitly by a previous step in the scenario, or**
 - **A default app name set in a configuration file.**
- **This allows the reusable steps to be used to drive multiple apps within the same test.**

AC written in UI-style using reusable steps

- I began copying the Gherkin text from completed tests back into the Jira tickets. This showed the small group how the ticket's AC had been translated by Test into executable Gherkin.
- I then asked the small group which style they preferred: the original AC's style, or the executable Gherkin style with its extensive use of the reusable UI-oriented steps that was a bit like a traditionally-written manual test with its detailed UI-oriented test steps.
- Everybody agreed that the Gherkin-style (with its manual-test-style approach) was better:
 - Product preferred it because it described the requirement's behavior in terms of the application's UI, which is how Product tend to think of things.
 - Dev preferred it because it showed concrete examples of how their code should behave, thus removing any ambiguity with Product's intentions. The concrete examples also gave agreed names to the various UI controls, so Dev would code to those names, and Test would use those names in their tests.
 - Test preferred it because the translation from AC to executable Gherkin was easy, almost trivial.
 - Scrum master and project management preferred it because the concrete examples seemed to help limit ticket scope creep (Dev only implemented code if the AC contained a concrete example that would test that code).
- Hence, the AC in subsequent tickets was authored by the small group in a more Gherkin-friendly, UI-oriented style, based around the reusable step functions, and including concrete, valid and meaningful data values.
- In other words, the AC became written primarily from a Test perspective, whereas in many projects (from my experience) Test is more of an afterthought when it comes to AC.

Benefits of this approach

Observed benefits with this approach include:

- Non-programmers can develop the application's automated test cases with little training.
- Automated tests can be developed rapidly due to test-friendly AC.
- Library of reusable steps can be built over time in parallel with test cases.
- Sprint success improved, due to ticket scope being limited to only the concrete scenarios stated in ticket's AC.
- BDD / Gherkin is compatible with many test automation tools and programming languages. You can find a few combinations demonstrated at my personal GitHub page (see <https://github.com/timbwfox>).
- Gherkin itself is tool-stack neutral, and so the test cases (i.e. the .feature files) would not need to change due to a transition from (say) Selenium to Playwright (although the step definition functions typically would).

Can AI tools help?

- The project discussed in this presentation occurred before AI code-generation tools became widely available.
- One of the main reasons for creating the BDD test automation demos at <https://github.com/timbwfox> is to show that AI tools (namely GitHub Co-Pilot and Claude Code) are conversant with BDD, the various BDD tool-stacks and are compatible with the approach described in this presentation.
- All code, files, READMEs, CI/CD pipelines, etc. in all the repos at <https://github.com/timbwfox>, including all maintenance edits and automated test cases, were 100% generated/edited by the AI tools under my chat prompts.
- So, yes, the latest AI tools are definitely compatible with this approach.
- What wasn't tested was whether the AI tools can help with the creation/maintenance of AC text within (say) Jira tickets, and/or whether the AI tools can generate automated test cases directly from the AC.
- My GitHub repos do demonstrate AI tools can generate UI-oriented, BDD-based, automated tests for an existing application (given access to that application's source code), but they do not demonstrate automated test case generation direct from the AC.
- From what I've read online that should be possible directly within Jira's AI tools, and it would be interesting to try that out on a real project.

Test/QA Personnel Count

- The Test personnel count on the 6-month project discussed in this presentation was 4 SDET (1 in-house lead, 3 contractors including myself), plus 3 x 2-week sessions from manual testers.
- But the in-house lead SDET was moved to another project mid-way through (which was when I became the effective test lead), so really 3.5 SDET.
- I introduced the UI-oriented, reusable test step approach, and implemented most functions in the reusable step library, and I became the test representative in the small group when in-house lead SDET moved to another project.
- I also introduced the UI-centric, reusable step approach to AC leading to approach being adopted project-wide, and I also taught the team (including manual testers) how to develop the automated test cases using this approach.
- The other 2 SDETs and manual testers concentrated mainly on developing the automated test cases.
- But, based on my experience using GitHub Co-Pilot and Claude Code in my personal AI projects, I firmly believe that an SDET familiar with this approach and using the latest AI tools would be able to single handedly:
 - Design/create all functions in the reusable step definitions library.
 - Configure Jira to create all (or at least the majority) of automated test cases directly from the AC in Jira tickets.
 - Update the CI/CD pipelines to run the automated tests on (say) Git pull requests and branch merges.
 - Be an integral part of the ticket grooming process, particularly relating to AC and testing.